INVENTORY MANAGEMENT AND E-COMMERCE WEBSITE - SR FAB

A PROJECT REPORT

Submitted by

SRIRAM M

23ADR161

RAMYA G

23ADR138

SIVAPRAKASH M P

23ADR153

in partial fulfilment of the requirements for

the award of the degree of

BACHELOR OF TECHNOLOGY IN ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

KONGU ENGINEERING COLLEGE

TRANSFORM YOURSELF

(Autonomous)

PERUNDURAI ERODE – 638 060

MARCH 2026

INVENTORY MANAGEMENT AND E-COMMERCE WEBSITE - SR FAB

A PROJECT REPORT

Submitted by

SRIRAM M

23ADR161

RAMYA G

23ADR138

SIVAPRAKASH M P

23ADR153

in partial fulfilment of the requirements for

the award of the degree of

BACHELOR OF TECHNOLOGY IN ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

KONGU ENGINEERING COLLEGE

TRANSFORM YOURSELF

(Autonomous)

PERUNDURAI ERODE – 638 060

MARCH 2026

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

KONGU ENGINEERING COLLEGE

(Autonomous)

PERUNDURAI ERODE – 638 060

MARCH 2026

BONAFIDE CERTIFICATE

This is to certify that the Project report entitled **INVENTORY MANAGEMENT AND E-COMMERCE WEBSITE - SR FAB** is the bonafide record of the project work done by **SRIRAM M(23ADR161), RAMYA G(23ADR138), SIVAPRAKASH M P(23ADR153)** in the partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Artificial Intelligence and Data Science of Anna University Chennai during the year 2025-2026.

SUPERVISOR **HEAD OF THE DEPARTMENT**

(Signature with seal)

Date:

Submitted for the end semester viva voce examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

KONGU ENGINEERING COLLEGE

(Autonomous)

PERUNDURAI ERODE – 638 060

MARCH 2026

DECLARATION

We affirm that the Project Report titled **INVENTORY MANAGEMENT AND E-COMMERCE WEBSITE - SR FAB** submitted in partial fulfillment of the requirements for the award of Bachelor of Technology is the original work carried out by us. It has not formed the part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion this or any other candidate.

Date:

SRIRAM M
(23ADR161)

RAMYA G
(23ADR138)

SIVAPRAKASH M P
(23ADR153)

I certify that the declaration made by the above candidate is true to the best of my knowledge.

Date: **Name and Signature of the Supervisor with Seal**

ACKNOWLEDGEMENT

We express our sincere gratitude to **Thiru. E. R. K. KRISHNAN, M.Com.,** our beloved Correspondent, and all other philanthropic trust members of Kongu Vellalar Institute of Technology Trust who have always encouraged us in academic and co- curricular activities.

We are extremely thankful with no words of a formal nature to the dynamic Principal **Dr. R. PARAMESHWARAN, M.E., Ph.D.,** for providing the necessary facilities to complete our work.

We would like to express our sincere gratitude to our respected Head of the Department **Dr. P.NATESAN, M.E., Ph.D.,** for providing the necessary facilities.

We extend our thanks to **Mrs. P. RAMYA M.E(CSE).,** Assistant Professor, Artificial Intelligence and Data Science, Project Coordinator for her encouragement and valuable advice that made us carry out the project work successfully.

We extend our gratitude to our Supervisor **Dr. M. VIMALADEVI, M.E., Ph.D.,** Associate Professor of Artificial Intelligence and Data Science, for her valuable ideas and suggestions, which have been very helpful in the project. We are grateful to all the faculty members of the Artificial Intelligence and Data Science Department, for their support.

ABSTRACT

The **“Inventory Management and E-Commerce Website - SR FAB”** is a web-based platform designed to streamline the process of purchasing raw fabrics, converting them into finished garments, and selling them through both offline and online channels. The system replaces traditional manual record-keeping and disconnected sales processes with an integrated digital solution that manages inventory, product display, order processing, and customer interactions efficiently.

Built on a three-tier architecture consisting of a user-friendly browser-based frontend, a robust application server, and a secure database, the platform enables Admin, Staff, and Customers to interact through role-based access. It maintains complete records of fabric procurement, garment production stages, stock availability, and sales transactions, ensuring real-time visibility of products from manufacturing to delivery.

The website provides customers with an easy-to-use interface to explore various garment collections, place orders online, and track their purchases, while also supporting offline sales operations. The system aims to enhance business growth by improving operational efficiency, reducing manual errors, and increasing market reach through an online presence. Additionally, it lays the foundation for future integration with inventory management, payment systems, and analytics tools, enabling smarter decision-making and scalable business expansion.

TABLE OF CONTENTS

CHAPTER No.	TITLE	PAGE No.
	ABSTRACT	**VI**
	LIST OF TABLES	**IX**
	LIST OF FIGURES	**X**
1.	**INTRODUCTION**	**11**
	1.1 OVERVIEW OF THE PROJECT	11
	1.2 OBJECTIVES	13
	1.3 SYSTEM STUDY	13
	1.4 PROBLEM DESCRIPTION	14
2.	**SYSTEM ANALYSIS**	**16**
	2.1 EXISTING SYSTEM	16
	2.2 PROPOSED SYSTEM	16
	2.3 HIGH LEVEL ARCHITECTURE DIAGRAM	**18**
	2.4 ER DIAGRAM	**19**
	2.5 DATAFLOW DIAGRAM	**20**
	2.5.1 USE CASE DIAGRAM	**20**
	2.5.2 SEQUENCE DIAGRAM	**21**
	2.6 USER JOURNEY AND WORKFLOWS	**21**
	2.7 SYSTEM REQUIREMENTS	**22**
	2.7.1 FUNCTIONAL REQUIREMENTS	**22**
	2.7.2 NON FUNCTIONAL REQUIREMENTS	**23**

3.	**SYSTEM DESIGN**	**26**
	3.1 BUSINESS RULES AND LOGIC	26
	3.2 INPUT DESIGN	27
	3.3 DATABASE DESIGN	28
	3.4 MODULE DESCRIPTION	28
	3.5 OUTPUT DESIGN	29
4.	**SYSTEM TESTING**	**31**
5.	**CONCLUSION AND FUTURE WORK**	**33**
	APPENDIX 1	**34**
	APPENDIX 2	**67**
	REFERENCES	**69**
	MAPPING OF PROJECT GOALS TO SDG	

LIST OF TABLES

TABLE No.	TABLE NAME	PAGE No.
Table 3.1	**APPOINTMENT & ENQUIRY RULES**	16
Table 3.2	**SERVICE INFORMATION RULES**	17
Table 3.3	**WEBSITE INTERACTION RULES**	17
Table 3.4	**CONTACT & COMMUNICATION RULES**	18

FIGURE No.	FIGURE NAME	PAGE No.
Fig 2.1	HIGH LEVEL ARCHITECTURE	18
Fig 2.2	ER DIAGRAM	19
Fig 2.3	USE CASE DIAGRAM	10
Fig 2.4	SEQUENCE DIAGRAM	21
Fig 2.5	USER FLOW DIAGRAM	22

CHAPTER 1

INTRODUCTION

Garment manufacturing companies play an important role in the textile and fashion industry by producing a wide range of clothing products for customers and retailers. In many small and medium-scale garment factories, operational activities such as inventory management, production tracking, and shipment handling are often maintained manually using registers or basic spreadsheets. These traditional methods can lead to errors in stock calculation, difficulty in maintaining accurate records, and delays in monitoring production and shipment activities.

As the volume of raw materials, work-in-progress items, and finished garments increases, it becomes more challenging for management to efficiently track inventory levels and manage logistics operations. Manual processes also make it difficult to monitor product availability, track shipments, and generate reports related to production and sales.

To address these challenges, the **Fabric Flow System** is proposed as a software-based solution that helps manage inventory, production flow, shipment tracking, and online selling in a garment manufacturing company. The system allows administrators to maintain product details, monitor stock levels, track orders, and update shipment status through a centralized digital platform.

1.1 OVERVIEW OF THE PROJECT

Garment manufacturing companies play an important role in the textile and fashion industry by producing a wide range of clothing products for customers and retailers. In many small and medium-scale garment factories, operational activities such as inventory management, production tracking, and shipment handling are often maintained manually using registers or basic spreadsheets. These traditional methods can lead to errors in stock calculation, difficulty in maintaining accurate records, and delays in monitoring production and shipment activities.

As the volume of raw materials, work-in-progress items, and finished garments increases, it becomes more challenging for management to efficiently track inventory levels and manage logistics operations. Manual processes also make it difficult to monitor product availability, track shipments, and generate reports related to production and sales.

To address these challenges, the **Fabric Flow System** is proposed as a software-based solution that helps manage inventory, production flow, shipment tracking, and online selling in a garment manufacturing company. The system allows administrators to maintain product details, monitor stock levels, track orders, and update shipment status through a centralized digital platform.

1.2 OBJECTIVES

The primary objective of the **Fabric Flow System** is to develop a digital platform that simplifies the management of inventory, production flow, and shipment tracking in a garment manufacturing company. The system aims to improve operational efficiency, reduce manual errors, and provide better visibility of materials and finished products.

The specific objectives of the project are:

• To develop a centralized system for managing **raw materials, work-in-progress items, and finished garments**.

• To provide a **user-friendly interface** for managing inventory and product details efficiently.

• To maintain **accurate stock information** and provide real-time updates about inventory availability.

• To implement a **shipment tracking system** that records shipment details and delivery status.

• To support **online selling of garments**, allowing customers to view products and place orders easily.

• To develop an **admin dashboard** that allows management to add, update, and monitor product and inventory details.

• To generate **reports related to inventory, orders, and shipments** for better business decision-making.

• To design a **reliable and scalable system** that supports the growing needs of the garment manufacturing business.

1.3 SYSTEM STUDY

A detailed A detailed system study was conducted to understand the existing operations

of the garment manufacturing company **SR FAB**. The study focused on how raw materials are managed, how production activities are carried out, and how finished garments are prepared and shipped to customers.

In many garment manufacturing units, operational activities such as raw material procurement, inventory management, and shipment tracking are often maintained using manual registers or basic spreadsheets. Workers record stock details, production information, and shipment records manually. This process is time-consuming and may lead to errors, inaccurate stock information, and difficulty in tracking materials and finished products.

The study also revealed that managers face challenges in monitoring inventory levels and tracking shipment status in real time. Without a centralized system, it becomes difficult to maintain accurate records and ensure timely delivery of products to clients.

Additionally, the company currently lacks a proper online selling platform to showcase garments and manage customer orders digitally. This limits the company's ability to reach more customers and expand its business.

1.4 PROBLEM DESCRIPTION

The traditional system used in many garment manufacturing companies faces several operational challenges that affect efficiency and management control. In companies like SR FAB, many activities related to inventory management, production tracking, and shipment handling are often maintained manually using registers or basic spreadsheets.

In the existing system, raw material stock, work-in-progress items, and finished garments are recorded manually. This method can lead to errors, inaccurate stock information, and difficulty in tracking materials throughout the production process. It also becomes challenging for managers to monitor inventory levels and ensure that materials are available when needed for production.

Shipment tracking is another major challenge in the traditional system. Shipment details and delivery status are often maintained using manual records, making it difficult to track

the movement of goods and ensure timely delivery to customers.

Additionally, the absence of a centralized digital platform limits the company's ability to manage orders and inventory efficiently. Without proper system integration, it becomes difficult to generate reports, monitor stock availability, and manage logistics operations effectively.

Furthermore, the lack of an online selling platform restricts the company from reaching a wider customer base and promoting its products digitally.

CHAPTER 2

SYSTEM ANAYLSIS

2.1 EXISTING SYSTEM

The traditional system used in many garment manufacturing companies faces several operational challenges that affect efficiency and management control. In companies like SR FAB, many activities related to inventory management, production tracking, and shipment handling are often maintained manually using registers or basic spreadsheets.

In the existing system, raw material stock, work-in-progress items, and finished garments are recorded manually. This method can lead to errors, inaccurate stock information, and difficulty in tracking materials throughout the production process. It also becomes challenging for managers to monitor inventory levels and ensure that materials are available when needed for production.

Shipment tracking is another major challenge in the traditional system. Shipment details and delivery status are often maintained using manual records, making it difficult to track the movement of goods and ensure timely delivery to customers.

Additionally, the absence of a centralized digital platform limits the company's ability to manage orders and inventory efficiently. Without proper system integration, it becomes difficult to generate reports, monitor stock availability, and manage logistics operations effectively.

Furthermore, the lack of an online selling platform restricts the company from reaching a wider customer base and promoting its products digitally.

2.2 PROPOSED SYSTEM

The proposed system introduces a **software-based Fabric Flow platform** that helps manage inventory, production flow, shipment tracking, and online selling for a garment manufacturing company. The system replaces manual record-keeping with a centralized digital solution that improves accuracy,

efficiency, and management control.

The platform provides an **inventory management module** where raw materials, work-in-progress items, and finished garments can be recorded and monitored. This allows the company to maintain accurate stock information and view real-time inventory updates.

The system also includes a **shipment tracking module** that records shipment details and updates delivery status such as *In Transit* and *Delivered*. This feature helps management track product movement and ensure timely delivery to customers.

In addition, the system supports **online selling of garments**. Customers can view available products, check details, and place orders through the platform. This helps the company expand its market reach and improve customer accessibility.

For administrative purposes, the system provides an **admin dashboard** where authorized users can manage product details, update inventory records, track orders, and generate reports related to stock and shipments.

2.3 HIGH LEVEL ARCHITECTURE DIAGRAM

The High-Level Architecture Diagram illustrates the overall structure of the Fabric Flow System, including the frontend, backend, and database components. It shows how users interact with the system through the interface and how data flows between different layers. This architecture ensures efficient communication, data management, and smooth system operation.

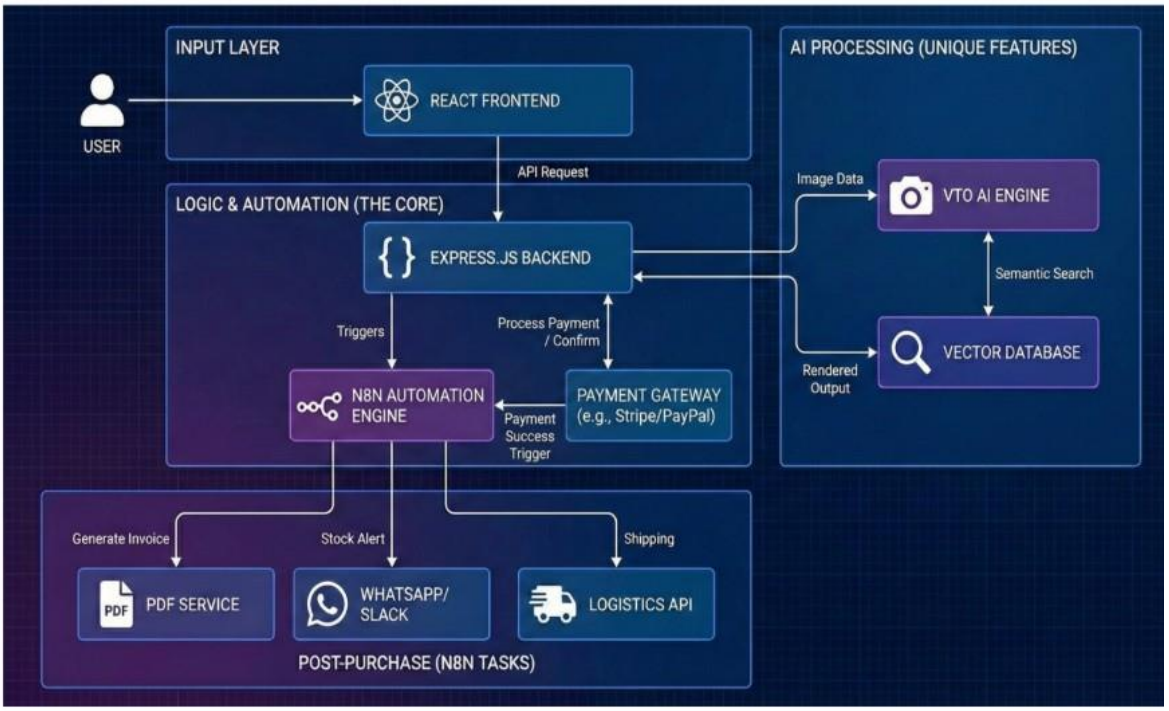


FIGURE 2.1 ARCHITECTURE

2.4 ER DIAGRAM

The Entity Relationship (ER) Diagram represents the database structure of the Fabric Flow System. It shows the main entities such as User, Product, Inventory, Order, and Shipment along with their attributes and relationships. The diagram helps in understanding how data is organized and connected within the system. It ensures efficient data storage, retrieval, and management.

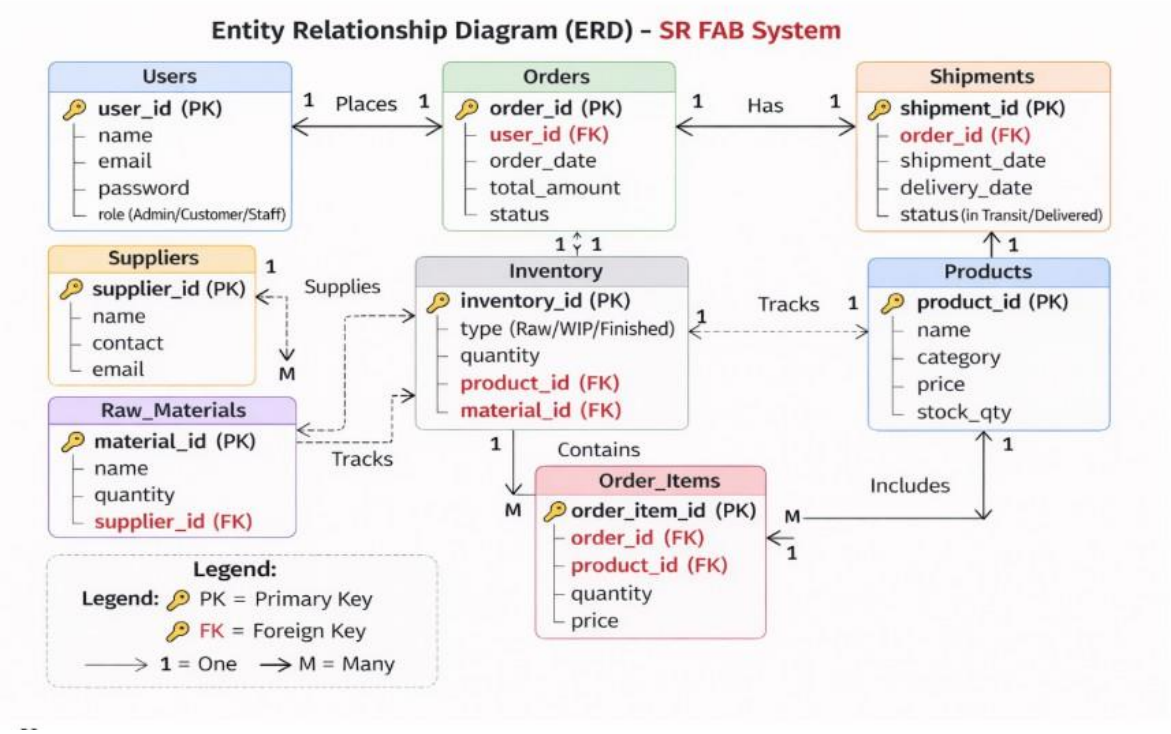


FIGURE 2.2 ER DIAGRAM

2.5 DATAFLOW DIAGRAM

The Data Flow Diagram (DFD) represents how data moves within the Fabric Flow System. It shows the interaction between external entities such as Customer and Admin, system processes like inventory management, order processing, and shipment tracking, and data stores.

2.5.1 USE CASE DIAGRAM

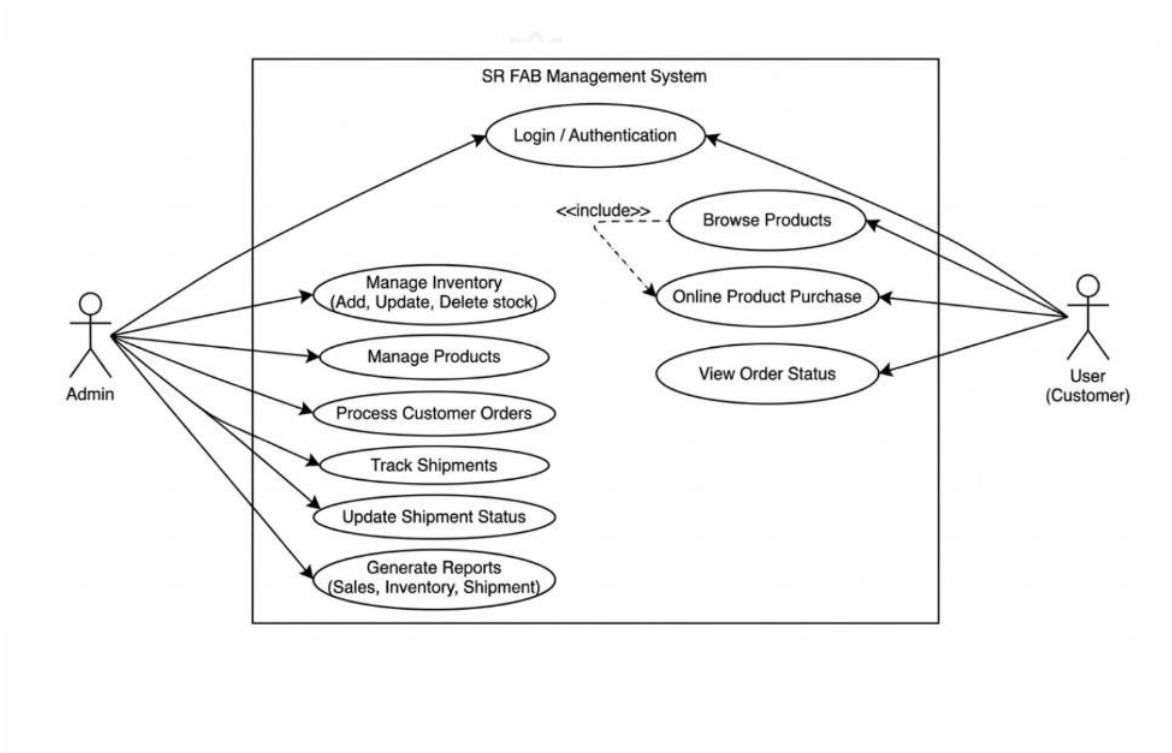


FIGURE 2.3 FABRICFLOW SYSTEM USECASE DIAGRAM

2.5.2 SEQUENCE DIAGRAM

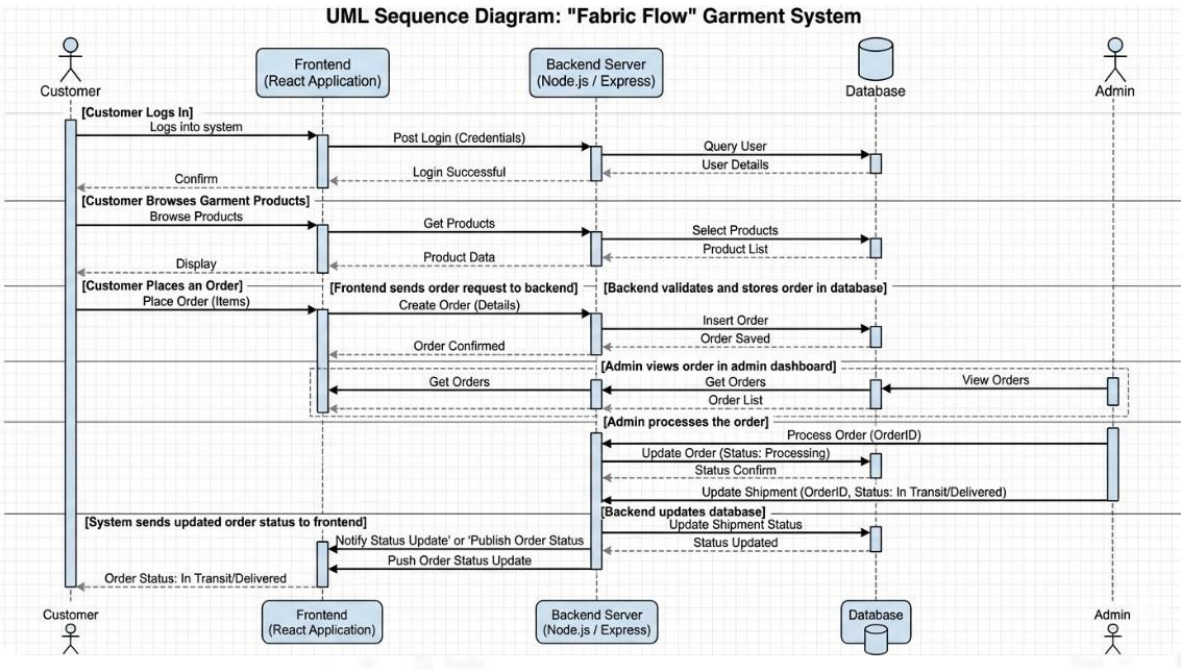


FIGURE 2.4 SEQUENCE DIAGRAM OF FABRIC FLOW SYSTEM

2.6 USER JOURNEY AND WORKFLOWS

The workflow of the **Fabric Flow System** explains how customers interact with the platform and how inventory, orders, and shipments are managed by the administrator. The process begins when customers access the platform to explore available garment products.

Customers can browse the product catalog, view product details such as garment type, description, and price, and place orders through the system. Once the order is placed, the system records the order details and stores them in the database.

The order information is then sent to the **admin dashboard**, where the administrator can review the order and check product availability in the inventory. The admin processes the order and updates the shipment details.

After processing, the administrator updates the shipment status such as **In Transit** or **Delivered**. Customers can view the order status and receive updates regarding the delivery.

.

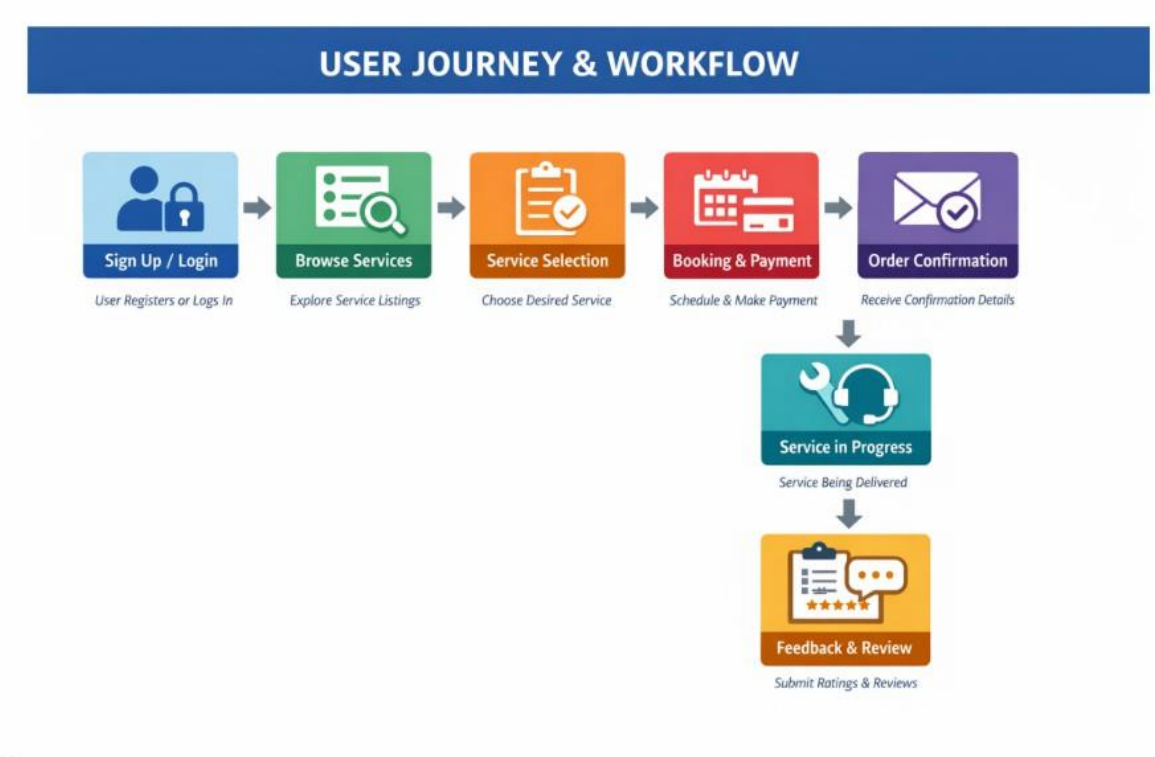


FIGURE 2.5 USER FLOW DIAGRAM

2.7 SYSTEM REQUIREMENTS

2.7.1 FUNCTIONAL REQIUREMENTS

1. User Authentication

The system should allow users to register and log in securely. Authentication ensures that only authorized users can access the system. Administrators will have full access to manage inventory, products, and shipments, while customers can browse products and place orders.

2. Product Management

The system should allow the administrator to add new garment products and manage existing product details. The admin can enter information such as product name, description, price, and category. The system should also allow updating product details when there are changes and deleting products that are no longer available.

3. Inventory Management

The system should maintain accurate inventory records for raw materials, work-in-progress items, and finished garments. It should allow administrators to update

stock quantities whenever materials are used or products are sold. This helps the management monitor stock availability and manage inventory efficiently.

4. Order Management

The system should allow customers to browse garment products and place orders online. Once an order is placed, the system should record order details and store them in the database. Administrators should be able to view, confirm, and manage customer orders through the admin dashboard.

5. Shipment Tracking

The system should allow administrators to create and manage shipment records. Shipment status such as **Pending, In Transit, and Delivered** can be updated to track the movement of products. This helps ensure timely delivery and better logistics management.

6. Stock Monitoring

The system should allow administrators to view real-time stock availability of products and materials. This feature helps management maintain proper stock levels and avoid shortages during production

7. Report Generation

The system should generate reports related to inventory, orders, and shipments. These reports help management analyze business performance and make better decisions regarding production and inventory control.

8. Search and Filter Products

The system should allow users to search for products easily using product names or categories. It should also provide filtering options to quickly find specific garments during browsing or management operations

2.7.2 NON- FUNCTIONAL REQUIREMENTS

1. Performance

The system should provide fast and efficient responses during operations such as product browsing, order placement, inventory updates, and shipment tracking. Even when multiple users access the system at the same time, it should process requests quickly and update data without delays. This ensures smooth system operation and efficient management of garment inventory and orders.

2. Usability

The system should have a simple, clean, and user-friendly interface so that administrators and users can operate it easily without requiring advanced technical knowledge. Important functions such as product management, inventory tracking, order processing, and shipment updates should be easily accessible through clear navigation menus.

3. Reliability

The system should function consistently without frequent crashes or failures. All product information, inventory records, order details, and shipment data must be stored correctly and should remain accessible whenever required. The system must ensure that data is updated accurately after every operation.

4. Security

The system should provide secure login authentication for authorized users such as administrators. Unauthorized users should not be able to access or modify system data. Sensitive information such as product details, order records, and inventory data must be protected from misuse or unauthorized access.

5. Scalability

The system should be capable of handling future growth in the garment manufacturing business. As the number of products, orders, and users increases, the system should still perform efficiently. The system design should also support the addition of new features in the future.

6. Maintainability

The system should be designed using a modular and well-structured architecture so that developers can easily maintain and update the system. Bug fixes, performance improvements, or new feature additions should be implemented without affecting existing functionalities.

7. Compatibility

The system should work smoothly on common web browsers such as Google Chrome, Microsoft Edge, and Mozilla Firefox. It should run efficiently on standard computers without requiring high-end hardware or special software installations..

8. Data Backup and Recovery

The system should store data securely in a database and ensure that important information such as product details, inventory records, order history, and shipment data is not lost. Backup mechanisms should be available to recover data in case of system failure or unexpected issues.

CHAPTER 3
SYSTEM DESIGN

3.1 BUSINESS RULES AND LOGIC

Business rules define the guidelines and conditions that control how the **Fabric Flow System** operates and how data is processed within the garment manufacturing environment. These rules ensure proper system functionality, data accuracy, and efficient management of inventory, orders, and shipments.

1. Product Management Rules

The system should allow the administrator to add, update, and delete garment product details such as product name, category, description, price, and stock quantity. Each product must have a unique identifier to avoid duplication and ensure proper tracking within the system.

2. Order Management Rules

Customers should be able to browse products and place orders through the system. Once an order is placed, it must be recorded in the database with details such as customer information, order date, and order status. Each order should have a unique order ID for tracking purposes.

3. Inventory Update Rules

Whenever an order is placed or materials are used in production, the system must automatically update the inventory levels. This ensures accurate tracking of raw materials, work-in-progress items, and finished garments.

4. Stock Availability Rules

The system should not allow customers to place orders for products that are out of stock. If the requested quantity exceeds available stock, the system should notify the administrator or user and prevent the order until the stock is updated.

5. Shipment Tracking Rules

The system should allow the administrator to update shipment status such as

Pending, In Transit, and Delivered. Each shipment must be linked to a specific order, and updates should be reflected in real time for tracking purposes.

6. Data Security Rules

Only authorized users (administrators) should be able to access and manage system data through secure login authentication. Unauthorized users must not be allowed to modify or access sensitive data such as inventory, orders, or shipment details.

7. Reporting and Analysis Rules

The system should generate reports related to inventory, orders, and shipments. These reports help management analyze business performance and make better decisions regarding production, stock management, and logistics.

3.2 INPUT DESIGN

Input design refers to the process of designing how data is entered into the **Fabric Flow System**. It ensures that the data provided by users is accurate, complete, and easy to input. A well-designed input system helps reduce errors and improves overall system efficiency.

In the Fabric Flow System, various input forms are provided for managing different operations such as **user authentication, product management, inventory updates, order placement, and shipment tracking**.

The system includes forms such as **login forms, product entry forms, inventory update forms, order forms, and shipment update forms**. In the product management section, the administrator can enter details such as product name, category, description, price, and available stock using structured input fields. Proper validation is applied to ensure that only valid data is entered, such as numeric values for price and quantity and mandatory fields for essential information.

3.3 DATABASE DESIGN

The **Fabric Flow System** uses a database to store and manage all application data related to garment inventory, orders, and shipment tracking. The database maintains information such as product details, inventory records, customer information, order details, and shipment status.

The system stores **product data** including product name, description, category, price, and stock availability. The **inventory database** keeps track of raw materials, work-in-progress items, and finished garments, ensuring accurate stock management.

When a customer places an order, the system records the **order details** such as customer information, selected products, quantity, order date, and order status. The database also maintains **shipment data**, including shipment ID, delivery status, and tracking information.

Whenever an order is placed or inventory is updated, the system automatically reflects the changes in the database to maintain consistency and accuracy. This helps in real-time tracking of stock levels and order status.

3.4 MODULE DESCRIPTION

3.4.1 MODULE 1

This module is responsible for managing all **garment product and inventory-related information** in the Fabric Flow System. The administrator can add new products by entering details such as product name, category, description, price, and available stock quantity. The system also allows updating product details whenever there are changes in price, stock levels, or product information. Products that are no longer available can be removed from the system.

The inventory management part of this module ensures that stock levels are maintained accurately for **raw materials, work-in-progress items, and finished garments**. Whenever a product is sold or materials are used in production, the system automatically updates the stock quantity in the database.

This module helps the administrator monitor product availability in real time and prevents issues such as overstocking or stock shortages. It also improves overall inventory control by replacing manual tracking with an efficient digital system, ensuring accuracy and better decision-making.

3.4.2 MODULE 2

Order Management

This module handles the complete **order processing workflow** in the Fabric Flow System. Customers can browse available garment products, select items, and place orders through the system. The order details such as product information, quantity, customer details, and order date are stored in the database.

The administrator can view all customer orders through the **admin dashboard**. The system allows the admin to verify the order, check product availability, and update the order status accordingly. Each order is assigned a unique order ID, which helps in tracking and managing orders efficiently.

3.4.3 MODULE 3

Shipment Tracking

This module is responsible for managing the **shipment and delivery process** of orders. Once an order is processed, the administrator can create a shipment record and update the delivery status.

The system allows the admin to update shipment status such as **Pending, In Transit, and Delivered**. Each shipment is linked to a specific order, enabling easy tracking of product movement from the company to the customer.

This module improves logistics management by providing real-time updates on shipment status and ensuring timely delivery of products.

.

3.5 OUTPUT DESIGN

Output design refers to the way information is presented to users after

processing the data in the system. The main goal of output design is to provide clear, accurate, and useful information to both customers and administrators in an easy-to-understand format.

In the **Fabric Flow System**, outputs are displayed through various screens such as product listings, order details, inventory status, and shipment tracking information.

The system provides a **product display interface** where customers can view garment details such as product name, description, price, and availability. This helps users make informed decisions while placing orders.

CHAPTER 4
SYSTEM TESTING

4.1. Functional Testing

Functional testing ensures that every feature of the **Fabric Flow System** works according to the specified requirements. This includes verifying that garment products can be added, updated, and deleted correctly, inventory levels are updated properly, customer orders are processed accurately, and shipment tracking information is displayed correctly.

The testing process also checks input validation, such as preventing invalid product details, ensuring correct quantity values, and confirming that orders cannot be placed for out-of-stock products. Functional testing also verifies proper error handling for situations such as incorrect login credentials or invalid order details.

By testing each module individually, functional testing ensures that the system behaves as expected and supports smooth management of inventory, orders, and shipment operations.

4.2. Integration Testing

Integration testing evaluates how different modules of the system work together as a complete system. For example, when a customer places an order, the system should automatically update the inventory and create an order record in the database.

This testing ensures proper communication between system components such as the **user interface, backend server, and database**. It verifies that data flows correctly between modules like product management, inventory tracking, order processing, and shipment management.

Integration testing helps identify issues such as incorrect inventory updates, mismatched order data, or delayed shipment status updates.

4.3. **User Acceptance Testing (UAT)**

User Acceptance Testing involves real users such as administrators and system operators to ensure that the system meets practical business requirements. During

this stage, users interact with the system to verify that tasks such as product management, order processing, inventory monitoring, and shipment tracking are simple and efficient.

UAT also evaluates the overall usability of the system, including ease of navigation, clarity of information, and responsiveness of the interface. Successful completion of UAT confirms that the system is user-friendly and suitable for real-world implementation.

4.4. **Performance Testing**

Performance testing evaluates the system's speed, responsiveness, and stability under different conditions. This includes verifying that the system can handle multiple users accessing the platform simultaneously and process operations such as product browsing, order placement, and shipment updates efficiently.

Performance testing also checks database response times, system load handling, and overall application performance during peak usage. The goal is to ensure that the Fabric Flow System operates smoothly and efficiently even as the number of users and transactions increases.

4.5. **Security Testing**

Security testing ensures that the system is protected from unauthorized access and potential data breaches. The Fabric Flow System implements secure login authentication so that only authorized administrators can manage product details, inventory records, and shipment information.

Security testing also verifies protection against common threats such as unauthorized data modification and improper access to sensitive system information. Proper security measures help maintain the confidentiality, integrity, and reliability of system data.

CHAPTER 5

CONCLUSION AND FUTURE WORK

Conclusion

The **Fabric Flow System** provides an effective digital solution for managing inventory, order processing, and shipment tracking in a garment manufacturing company. The system replaces manual record-keeping methods with an automated platform, reducing errors and improving data accuracy.

By digitizing operations, the system allows administrators to easily manage product details, monitor inventory levels, process customer orders, and track shipment status in real time. This improves operational efficiency and ensures better coordination between different business activities.

The system also enhances decision-making by providing clear visibility of stock levels, order details, and shipment information. Overall, the Fabric Flow System saves time, reduces manual effort, and provides better control over inventory and logistics operations, helping the company adapt to modern digital practices.

Future Work

In the future, the Fabric Flow System can be enhanced by adding advanced features to improve system functionality and user experience. A **mobile application** can be developed to allow administrators to manage inventory, orders, and shipments from anywhere.

Additional features such as **barcode or QR code scanning** can be integrated to simplify inventory management and speed up product tracking. The system can also include **online payment integration** to support secure and convenient transactions for customers.

Advanced technologies such as **AI-based demand prediction** can be implemented to analyze sales trends and forecast product demand, helping in better inventory planning. A **real-time analytics dashboard** can also be added to provide insights into sales, stock levels, and business performance.

APPENDIX 1

LOGIN PAGE:

```
import { useState } from 'react';
import { useAuth } from '../context/AuthContext';
import { useNavigate, Link } from 'react-router-dom';

const Login = () => {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const { login } = useAuth();
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    const success = await login(email, password);
    if (success) navigate('/');
  };

  return (
    <div className="flex min-h-[70vh] items-center justify-center py-12">
      <div className="w-full max-w-sm animate-fade-in">
        <div className="text-center">
          <h1 className="font-serif text-3xl font-bold text-primary">Welcome
Back</h1>
          <p className="mt-2 text-sm text-muted">Sign in to your SR FAB
account</p>
        </div>

        <form onSubmit={handleSubmit} className="mt-10 space-y-5">
          <div>
            <label className="mb-1.5 block text-xs font-semibold uppercase
tracking-widest text-primary">Email</label>
```

```
          <input
            type="email"
            value={email}
            onChange={(e) => setEmail(e.target.value)}
            required
            className="w-full border border-gray-200 px-4 py-3 text-sm
text-primary outline-none transition focus:border-accent"
            placeholder="you@example.com"
          />
        </div>
        <div>
          <label className="mb-1.5 block text-xs font-semibold uppercase
tracking-widest text-primary">Password</label>
          <input
            type="password"
            value={password}
            onChange={(e) => setPassword(e.target.value)}
            required
            className="w-full border border-gray-200 px-4 py-3 text-sm
text-primary outline-none transition focus:border-accent"
            placeholder="••••••••"
          />
        </div>
        <button
          type="submit"
          className="w-full bg-primary py-3.5 text-sm font-semibold
uppercase tracking-widest text-white transition hover:bg-accent"
        >
          Sign In
        </button>
      </form>

      <p className="mt-6 text-center text-sm text-muted">
        Don't have an account?{' '}
        <Link to="/register" className="font-medium text-accent transition
hover:text-primary">Create one</Link>
      </p>
    </div>
  </div>
```

```
  );
};

export default Login;
```

DASHBOARD CODE:

```
import { useEffect, useState } from 'react';
import api from '../api/axios';
import { toast } from 'react-toastify';
import { FiBox, FiGrid, FiList, FiTrash2, FiEdit3, FiPlus, FiTag, FiBarChart2,
FiAlertTriangle, FiEye, FiX } from 'react-icons/fi';
import { getProductImage } from '../utils/productImages';

const AdminDashboard = () => {
  const [categories, setCategories] = useState([]);
  const [products, setProducts] = useState([]);
  const [orders, setOrders] = useState([]);
  const [coupons, setCoupons] = useState([]);
  const [analytics, setAnalytics] = useState(null);
  const [activeTab, setActiveTab] = useState('analytics');

  // ── Form States ──

  // ── Form States ──
  const [showProductForm, setShowProductForm] = useState(false);
  const [showCategoryForm, setShowCategoryForm] = useState(false);
  const [showCouponForm, setShowCouponForm] = useState(false);

  const [editItem, setEditItem] = useState(null);
  const [viewOrder, setViewOrder] = useState(null);

  const emptyProduct = { productName: '', productDescription: '',
productPrice: '', productDiscount: '', brand: '', fabricType: '', productImages: '',
categoryId: '', variants: [{ size: '', color: '', sku: '', stock: '' }] };
  const emptyCategory = { categoryName: '', categoryDescription: '',
categoryImage: '' };
  const emptyCoupon = { code: '', discountPercent: '', minOrderAmount: '0',
maxDiscount: '0', active: true, validFrom: '', validUntil: '' };
```

```
  const [productForm, setProductForm] = useState(emptyProduct);
  const [categoryForm, setCategoryForm] = useState(emptyCategory);
  const [couponForm, setCouponForm] = useState(emptyCoupon);

  useEffect(() => { fetchData(); }, []);

  const fetchData = async () => {
    try {
      const [catRes, prodRes, orderRes, couponRes, analyticsRes] = await
Promise.all([
        api.get('/categories'),
        api.get('/products'),
        api.get('/orders').catch(() => ({ data: [] })),
        api.get('/admin/coupons').catch(() => ({ data: [] })),
        api.get('/admin/analytics').catch(() => ({ data: null })),
      ]);
      setCategories(catRes.data);
      setProducts(prodRes.data);
      setOrders(orderRes.data);
      setCoupons(couponRes.data);
      setAnalytics(analyticsRes.data);
    } catch (err) { console.error(err); }
  };

  // ── Image Upload ──
  // ── Product CRUD ──
  const handleProductSubmit = async (e) => {
    e.preventDefault();
    try {
      const payload = { ...productForm, productPrice:
parseFloat(productForm.productPrice) || 0, productDiscount:
parseFloat(productForm.productDiscount) || 0, variants:
productForm.variants.map(v => ({ ...v, stock: parseInt(v.stock) || 0 })) };
      if (editItem) {
        await
api.put(`/products/${editItem.productId}?categoryId=${productForm.category
Id}`, payload);
        toast.success('Product updated');
```

```
      } else {
        await api.post(`/products?categoryId=${productForm.categoryId}`,
payload);
        toast.success('Product added');
      }
      setShowProductForm(false); setEditItem(null);
setProductForm(emptyProduct); fetchData();
    } catch (err) { toast.error(err?.response?.data?.error || 'Failed'); }
  };

  const deleteProduct = async (id) => {
    if (!confirm('Delete this product?')) return;
    try { await api.delete(`/products/${id}`); toast.success('Deleted');
fetchData(); }
    catch { toast.error('Failed to delete'); }
  };

  const editProduct = (p) => {
    setProductForm({ productName: p.productName, productDescription:
p.productDescription || '', productPrice: p.productPrice, productDiscount:
p.productDiscount || 0, brand: p.brand || '', fabricType: p.fabricType || '',
productImages: p.productImages || '', categoryId: p.category?.categoryId || '',
variants: p.variants?.length ? p.variants : [{ size: '', color: '', sku: '', stock: '' }]
});
    setEditItem(p); setShowProductForm(true);
  };

  // ── Category CRUD ──
  const handleCategorySubmit = async (e) => {
    e.preventDefault();
    try {
      if (editItem) {
        await api.put(`/categories/${editItem.categoryId}`, categoryForm);
        toast.success('Category updated');
      } else {
        await api.post('/categories', categoryForm);
        toast.success('Category added');
      }
      setShowCategoryForm(false); setEditItem(null);
setCategoryForm(emptyCategory); fetchData();
```

```
    } catch (err) { toast.error(err?.response?.data?.error || 'Failed'); }
  };

  const deleteCategory = async (id) => {
    if (!confirm('Delete this category?')) return;
    try { await api.delete(`/categories/${id}`); toast.success('Deleted');
fetchData(); }
    catch { toast.error('Failed to delete'); }
  };

  // ── Coupon CRUD ──
  const handleCouponSubmit = async (e) => {
    e.preventDefault();
    try {
      const payload = { ...couponForm, discountPercent:
parseFloat(couponForm.discountPercent), minOrderAmount:
parseFloat(couponForm.minOrderAmount) || 0, maxDiscount:
parseFloat(couponForm.maxDiscount) || 0, validFrom:
couponForm.validFrom ? new Date(couponForm.validFrom).toISOString() :
null, validUntil: couponForm.validUntil ? new
Date(couponForm.validUntil).toISOString() : null };
      if (editItem) {
        await api.put(`/admin/coupons/${editItem.id}`, payload);
        toast.success('Coupon updated');
      } else {
        await api.post('/admin/coupons', payload);
        toast.success('Coupon created');
      }
      setShowCouponForm(false); setEditItem(null);
setCouponForm(emptyCoupon); fetchData();
    } catch (err) { toast.error(err?.response?.data?.error || 'Failed'); }
  };

  const deleteCoupon = async (id) => {
    if (!confirm('Delete this coupon?')) return;
    try { await api.delete(`/admin/coupons/${id}`); toast.success('Deleted');
fetchData(); }
    catch { toast.error('Failed'); }
  };
```

```
// ── Order Status ──
const updateOrderStatus = async (orderId, status) => {
  try {
    await api.put(`/orders/${orderId}/status`, { status });
    toast.success(`Order marked as ${status}`);
    fetchData();
  } catch { toast.error('Failed to update'); }
};

// ── Stock Update ──
const updateStock = async (variantId, stock) => {
  try {
    await api.put(`/products/variants/${variantId}/stock`, { stock:
parseInt(stock) });
    toast.success('Stock updated');
    fetchData();
  } catch { toast.error('Failed'); }
};

const addVariant = () => setProductForm({ ...productForm, variants:
[...productForm.variants, { size: '', color: '', sku: '', stock: '' }] });
const removeVariant = (i) => setProductForm({ ...productForm, variants:
productForm.variants.filter((_, idx) => idx !== i) });
const updateVariant = (i, field, val) => { const v = [...productForm.variants];
v[i] = { ...v[i], [field]: val }; setProductForm({ ...productForm, variants: v });
};

const tabs = [
  { id: 'analytics',  label: 'Analytics',   icon: FiBarChart2 },
  { id: 'products',   label: 'Products',    icon: FiBox },
  { id: 'categories', label: 'Categories',  icon: FiGrid },
  { id: 'orders',     label: 'Orders',      icon: FiList },
  { id: 'coupons',    label: 'Coupons',     icon: FiTag },
];

const statusColor = (s) => {
  if (!s) return 'bg-gray-100 text-gray-600';
  switch (s.toLowerCase()) {
    case 'delivered': return 'bg-green-100 text-green-700';
```

```
    case 'shipped': return 'bg-blue-100 text-blue-700';
    case 'cancelled': return 'bg-red-100 text-red-700';
    default: return 'bg-amber-100 text-amber-700';
  }
};

const inputClass = 'w-full border border-gray-200 px-3 py-2 text-sm
outline-none focus:border-accent bg-white';

return (
  <div className="mx-auto max-w-7xl animate-fade-in py-10">
    <h1 className="mb-8 font-serif text-3xl font-bold text-primary">Admin
Dashboard</h1>

    <div className="flex flex-col gap-8 lg:flex-row">
      {/* ── Sidebar ── */}
      <aside className="w-full lg:w-56 shrink-0">
        <nav className="space-y-1">
          {tabs.map((tab) => (
            <button key={tab.id} onClick={() => setActiveTab(tab.id)}
              className={`flex w-full items-center gap-3 px-4 py-3 text-sm
font-medium transition ${activeTab === tab.id ? 'bg-primary text-white
shadow-md' : 'bg-white text-muted hover:bg-gray-50 hover:text-primary'}`}>
              <tab.icon size={18} /> {tab.label}
            </button>
          ))}
        </nav>
      </aside>

      {/* ── Content ── */}
      <main className="flex-1 border border-gray-100 bg-white p-6
shadow-sm min-h-[60vh]">

        {/* ═══ ANALYTICS ═══ */}
        {activeTab === 'analytics' && (
          <div>
            <h2 className="mb-6 text-lg font-bold text-primary">Store
Analytics</h2>
            {analytics ? (
```

```
<>
  <div className="grid grid-cols-2 gap-4 lg:grid-cols-4">
    {[
      { label: 'Products', val: analytics.totalProducts, color: 'bg-blue-50
text-blue-700' },
      { label: 'Orders', val: analytics.totalOrders, color: 'bg-green-50
text-green-700' },
      { label: 'Revenue', val: `₹${Math.round(analytics.totalRevenue ||
0).toLocaleString('en-IN')}`, color: 'bg-amber-50 text-amber-700' },
      { label: 'Users', val: analytics.totalUsers, color: 'bg-purple-50
text-purple-700' },
    ].map((s) => (
      <div key={s.label} className={`p-5 ${s.color}`}>
        <p className="text-xs font-semibold uppercase
tracking-widest opacity-70">{s.label}</p>
        <p className="mt-1 text-2xl font-bold">{s.val}</p>
      </div>
    ))}
  </div>
  {analytics.recentOrders?.length > 0 && (
    <div className="mt-8">
      <h3 className="mb-4 text-sm font-bold uppercase
tracking-widest text-muted">Recent Orders</h3>
      <div className="space-y-2">
        {analytics.recentOrders.map(o => (
          <div key={o.id} className="flex items-center
justify-between border border-gray-50 p-3">
            <div>
              <span className="text-sm font-medium
text-primary">#{o.orderId}</span>
              <span className="ml-3 text-xs
text-muted">{o.user?.userName}</span>
            </div>
            <div className="flex items-center gap-3">
              <span className={`rounded-full px-2 py-1 text-[10px]
font-bold uppercase ${statusColor(o.status)}`}>{o.status}</span>
              {o.totalAmount > 0 && <span className="text-sm
font-semibold">₹{Math.round(o.totalAmount)}</span>}
            </div>
          </div>
```

```
                ))}
              </div>
            </div>
          )}
        </>
      ) : <p className="text-sm text-muted">Loading analytics...</p>}
    </div>
  )}

  {/* ═══ PRODUCTS ═══ */}
  {activeTab === 'products' && (
    <div>
      <div className="mb-6 flex items-center justify-between">
        <h2 className="text-lg font-bold text-primary">Manage Products
<span className="ml-2 rounded-full bg-gray-100 px-3 py-1 text-xs
font-semibold">{products.length}</span></h2>
        <button onClick={() => { setShowProductForm(true);
setEditItem(null); setProductForm(emptyProduct); }} className="flex
items-center gap-1 bg-primary px-4 py-2 text-xs font-semibold uppercase
tracking-widest text-white hover:bg-accent transition">
          <FiPlus size={14} /> Add Product
        </button>
      </div>

      {/* Product Form */}
      {showProductForm && (
        <form onSubmit={handleProductSubmit} className="mb-6 border
border-gray-100 p-5 space-y-4 bg-gray-50">
          <h3 className="font-semibold text-primary">{editItem ? 'Edit
Product' : 'New Product'}</h3>
          <div className="grid grid-cols-2 gap-4">
            <input placeholder="Product Name"
value={productForm.productName} onChange={e =>
setProductForm({...productForm, productName: e.target.value})}
className={inputClass} required />
            <select value={productForm.categoryId} onChange={e =>
setProductForm({...productForm, categoryId: e.target.value})}
className={inputClass} required>
              <option value="">Select Category</option>
              {categories.map(c => <option key={c.categoryId}
value={c.categoryId}>{c.categoryName}</option>)}
```

```
                </select>
                <input placeholder="Price (₹)" type="number"
value={productForm.productPrice} onChange={e =>
setProductForm({...productForm, productPrice: e.target.value})}
className={inputClass} required />
                <input placeholder="Discount (%)" type="number"
value={productForm.productDiscount} onChange={e =>
setProductForm({...productForm, productDiscount: e.target.value})}
className={inputClass} />
                <input placeholder="Brand" value={productForm.brand}
onChange={e => setProductForm({...productForm, brand: e.target.value})}
className={inputClass} />
                <input placeholder="Fabric Type"
value={productForm.fabricType} onChange={e =>
setProductForm({...productForm, fabricType: e.target.value})}
className={inputClass} />
              </div>
              <textarea placeholder="Description"
value={productForm.productDescription} onChange={e =>
setProductForm({...productForm, productDescription: e.target.value})}
className={inputClass + ' h-20'} />


              <input placeholder="Image URL"
value={productForm.productImages} onChange={e =>
setProductForm({...productForm, productImages: e.target.value})}
className={inputClass} />


              {/* Variants */}
              <div>
                <div className="flex items-center justify-between mb-2">
                  <span className="text-xs font-bold uppercase
text-muted">Variants</span>
                  <button type="button" onClick={addVariant}
className="text-xs text-accent font-semibold hover:text-primary">+ Add
Variant</button>
                </div>
                {productForm.variants.map((v, i) => (
                  <div key={i} className="grid grid-cols-5 gap-2 mb-2">
                    <input placeholder="Size" value={v.size} onChange={e =>
updateVariant(i, 'size', e.target.value)} className={inputClass} />
                    <input placeholder="Color" value={v.color} onChange={e =>
updateVariant(i, 'color', e.target.value)} className={inputClass} />
                    <input placeholder="SKU" value={v.sku} onChange={e =>
```

```
updateVariant(i, 'sku', e.target.value)} className={inputClass} />
                <input placeholder="Stock" type="number" value={v.stock}
onChange={e => updateVariant(i, 'stock', e.target.value)}
className={inputClass} />
                {productForm.variants.length > 1 && <button type="button"
onClick={() => removeVariant(i)} className="text-red-400
hover:text-red-600 text-xs">Remove</button>}
              </div>
            ))}
          </div>


          <div className="flex gap-2">
            <button type="submit" className="bg-primary px-6 py-2 text-xs
font-semibold uppercase tracking-widest text-white hover:bg-accent
transition">{editItem ? 'Update' : 'Create'}</button>
            <button type="button" onClick={() => {
setShowProductForm(false); setEditItem(null); }} className="border
border-gray-200 px-6 py-2 text-xs font-semibold uppercase tracking-widest
text-muted hover:text-primary transition">Cancel</button>
          </div>
        </form>
      )}


      {/* Product List */}
      <div className="space-y-3">
        {products.map(p => (
          <div key={p.productId} className="flex items-center
justify-between border border-gray-100 p-4 transition
hover:border-accent/50">
            <div className="flex items-center gap-4">
              <div className="h-12 w-12 overflow-hidden rounded
bg-gray-100">
                {p.productImages ? (
                  <img src={p.productImages.startsWith('http') ?
p.productImages : getProductImage(p.productId, 48, 48)}
alt={p.productName} className="h-full w-full object-cover" />
                ) : (
                  <div className="flex h-full w-full items-center justify-center
text-gray-300"><FiBox /></div>
                )}
              </div>
```

```
                <div>
                  <p className="font-medium
text-primary">{p.productName}</p>
                  <p className="text-sm
text-accent">₹{p.productPrice} {p.productDiscount > 0 && <span
className="ml-2 text-xs
text-green-600">(-{p.productDiscount}%)</span>}</p>
                  {/* Low stock badge */}
                  {p.variants?.some(v => v.stock <= 5) && (
                    <span className="inline-flex items-center gap-1 text-[10px]
text-red-600 font-semibold mt-1"><FiAlertTriangle size={10} /> Low
Stock</span>
                  )}
                </div>
              </div>
              <div className="flex items-center gap-2">
                <button onClick={() => editProduct(p)}
className="text-gray-400 transition hover:text-accent"><FiEdit3 size={16}
/></button>
                <button onClick={() => deleteProduct(p.productId)}
className="text-gray-400 transition hover:text-red-500"><FiTrash2
size={16} /></button>
              </div>
            </div>
          ))}
        </div>
      </div>
    )}

    {/* ═══ CATEGORIES ═══ */}
    {activeTab === 'categories' && (
      <div>
        <div className="mb-6 flex items-center justify-between">
          <h2 className="text-lg font-bold text-primary">Manage
Categories</h2>
          <button onClick={() => { setShowCategoryForm(true);
setEditItem(null); setCategoryForm(emptyCategory); }} className="flex
items-center gap-1 bg-primary px-4 py-2 text-xs font-semibold uppercase
tracking-widest text-white hover:bg-accent transition">
            <FiPlus size={14} /> Add Category
          </button>
```

```
        </div>

        {showCategoryForm && (
          <form onSubmit={handleCategorySubmit} className="mb-6
border border-gray-100 p-5 space-y-4 bg-gray-50">
            <h3 className="font-semibold text-primary">{editItem ? 'Edit
Category' : 'New Category'}</h3>
            <input placeholder="Category Name"
value={categoryForm.categoryName} onChange={e =>
setCategoryForm({...categoryForm, categoryName: e.target.value})}
className={inputClass} required />
            <textarea placeholder="Description"
value={categoryForm.categoryDescription} onChange={e =>
setCategoryForm({...categoryForm, categoryDescription: e.target.value})}
className={inputClass + ' h-20'} />

            <input placeholder="Image URL"
value={categoryForm.categoryImage} onChange={e =>
setCategoryForm({...categoryForm, categoryImage: e.target.value})}
className={inputClass} />

            <div className="flex gap-2">
              <button type="submit" className="bg-primary px-6 py-2 text-xs
font-semibold uppercase text-white hover:bg-accent transition">{editItem ?
'Update' : 'Create'}</button>
              <button type="button" onClick={() => {
setShowCategoryForm(false); setEditItem(null); }} className="border
border-gray-200 px-6 py-2 text-xs font-semibold uppercase text-muted
hover:text-primary transition">Cancel</button>
            </div>
          </form>
        )}

        <div className="grid grid-cols-2 gap-4 sm:grid-cols-3">
          {categories.map(c => (
            <div key={c.categoryId} className="relative flex flex-col
items-center border border-gray-100 p-6 text-center transition
hover:border-accent group">
              <span className="font-medium
text-primary">{c.categoryName}</span>
              {c.categoryDescription && <p className="mt-1 text-xs
text-muted">{c.categoryDescription}</p>}
              <div className="absolute top-2 right-2 opacity-0
```

```
group-hover:opacity-100 transition flex gap-1">
                <button onClick={() => { setEditItem(c); setCategoryForm({
categoryName: c.categoryName, categoryDescription: c.categoryDescription ||
", categoryImage: c.categoryImage || " }); setShowCategoryForm(true); }}
className="text-gray-400 hover:text-accent"><FiEdit3 size={14}
/></button>
                <button onClick={() => deleteCategory(c.categoryId)}
className="text-gray-400 hover:text-red-500"><FiTrash2 size={14}
/></button>
              </div>
            </div>
          ))}
        </div>
      </div>
    )}

    {/* ═══ ORDERS ═══ */}
    {activeTab === 'orders' && (
      <div>
        <h2 className="mb-6 text-lg font-bold text-primary">Manage
Orders</h2>
        <div className="overflow-x-auto border border-gray-100">
          <table className="w-full text-left text-sm">
            <thead className="bg-gray-50 text-xs uppercase text-muted">
              <tr>
                <th className="px-4 py-3 font-semibold">Order ID</th>
                <th className="px-4 py-3 font-semibold">Customer</th>
                <th className="px-4 py-3 font-semibold">Amount</th>
                <th className="px-4 py-3 font-semibold">Status</th>
                <th className="px-4 py-3 font-semibold">Action</th>
              </tr>
            </thead>
            <tbody className="divide-y divide-gray-100">
              {orders.map(o => (
                <tr key={o.id}>
                  <td className="px-4 py-3 font-medium
text-primary">#{o.orderId}</td>
                  <td className="px-4 py-3 text-muted">{o.user?.userName ||
'N/A'}</td>
                  <td className="px-4 py-3
```

```
font-medium">₹{Math.round(o.totalAmount || 0)}</td>
                <td className="px-4 py-3">
                  <span className={`inline-flex items-center rounded-full px-2
py-1 text-[10px] font-bold uppercase tracking-wide
${statusColor(o.status)}`}>{o.status || 'Pending'}</span>
                </td>
                <td className="px-4 py-3 flex gap-2">
                  <button onClick={() => setViewOrder(o)}
className="text-gray-400 hover:text-accent" title="View Details"><FiEye
size={16} /></button>
                  <select value={o.status || ''} onChange={e =>
updateOrderStatus(o.id, e.target.value)}
                    className="border border-gray-200 px-2 py-1 text-xs
bg-white outline-none w-28">
                    <option value="Order Placed">Placed</option>
                    <option value="Processing">Processing</option>
                    <option value="Shipped">Shipped</option>
                    <option value="Delivered">Delivered</option>
                    <option value="Cancelled">Cancelled</option>
                  </select>
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </div>
  )}

  {/* ═══ COUPONS ═══ */}
  {activeTab === 'coupons' && (
    <div>
      <div className="mb-6 flex items-center justify-between">
        <h2 className="text-lg font-bold text-primary">Manage
Coupons</h2>
        <button onClick={() => { setShowCouponForm(true);
setEditItem(null); setCouponForm(emptyCoupon); }} className="flex
items-center gap-1 bg-primary px-4 py-2 text-xs font-semibold uppercase
tracking-widest text-white hover:bg-accent transition">
          <FiPlus size={14} /> Create Coupon
```

```
          </button>
        </div>

        {showCouponForm && (
          <form onSubmit={handleCouponSubmit} className="mb-6 border
border-gray-100 p-5 space-y-4 bg-gray-50">
            <h3 className="font-semibold text-primary">{editItem ? 'Edit
Coupon' : 'New Coupon'}</h3>
            <div className="grid grid-cols-2 gap-4">
              <input placeholder="Code (e.g. SAVE20)"
value={couponForm.code} onChange={e =>
setCouponForm({...couponForm, code: e.target.value.toUpperCase()})}
className={inputClass} required />
              <input placeholder="Discount %" type="number"
value={couponForm.discountPercent} onChange={e =>
setCouponForm({...couponForm, discountPercent: e.target.value})}
className={inputClass} required />
              <input placeholder="Min Order ₹" type="number"
value={couponForm.minOrderAmount} onChange={e =>
setCouponForm({...couponForm, minOrderAmount: e.target.value})}
className={inputClass} />
              <input placeholder="Max Discount ₹" type="number"
value={couponForm.maxDiscount} onChange={e =>
setCouponForm({...couponForm, maxDiscount: e.target.value})}
className={inputClass} />
              <div><label className="text-xs text-muted">Valid
From</label><input type="date" value={couponForm.validFrom}
onChange={e => setCouponForm({...couponForm, validFrom:
e.target.value})} className={inputClass} /></div>
              <div><label className="text-xs text-muted">Valid
Until</label><input type="date" value={couponForm.validUntil}
onChange={e => setCouponForm({...couponForm, validUntil:
e.target.value})} className={inputClass} /></div>
            </div>
            <label className="flex items-center gap-2 text-sm"><input
type="checkbox" checked={couponForm.active} onChange={e =>
setCouponForm({...couponForm, active: e.target.checked})} />
Active</label>
            <div className="flex gap-2">
              <button type="submit" className="bg-primary px-6 py-2 text-xs
font-semibold uppercase text-white hover:bg-accent transition">{editItem ?
'Update' : 'Create'}</button>
              <button type="button" onClick={() => {
setShowCouponForm(false); setEditItem(null); }} className="border
```

```
border-gray-200 px-6 py-2 text-xs text-muted hover:text-primary
transition">Cancel</button>
            </div>
          </form>
        )}

        <div className="space-y-3">
          {coupons.map(c => (
            <div key={c.id} className="flex items-center justify-between
border border-gray-100 p-4">
              <div>
                <span className="font-mono text-sm font-bold
text-primary">{c.code}</span>
                <span className="ml-3 text-sm
text-accent">{c.discountPercent}% off</span>
                {c.minOrderAmount > 0 && <span className="ml-2 text-xs
text-muted">min ₹{c.minOrderAmount}</span>}
                {c.maxDiscount > 0 && <span className="ml-2 text-xs
text-muted">max ₹{c.maxDiscount}</span>}
                <span className={`ml-3 inline-block rounded-full px-2 py-0.5
text-[10px] font-bold ${c.active ? 'bg-green-100 text-green-700' : 'bg-gray-100
text-gray-500'}`}>{c.active ? 'Active' : 'Inactive'}</span>
              </div>
              <div className="flex gap-2">
                <button onClick={() => { setEditItem(c); setCouponForm({
code: c.code, discountPercent: c.discountPercent, minOrderAmount:
c.minOrderAmount || 0, maxDiscount: c.maxDiscount || 0, active: c.active,
validFrom: c.validFrom ? c.validFrom.slice(0, 10) : '', validUntil: c.validUntil
? c.validUntil.slice(0, 10) : '' }); setShowCouponForm(true); }}
className="text-gray-400 hover:text-accent"><FiEdit3 size={14}
/></button>
                <button onClick={() => deleteCoupon(c.id)}
className="text-gray-400 hover:text-red-500"><FiTrash2 size={14}
/></button>
              </div>
            </div>
          ))}
          {coupons.length === 0 && <p className="text-sm text-muted
text-center py-8">No coupons yet. Create one!</p>}
        </div>
      </div>
    )}
```

```
      </main>
    </div>

    {/* ── View Order Modal ── */}
    {viewOrder && (
      <div className="fixed inset-0 z-50 flex items-center justify-center
bg-black/50 p-4 backdrop-blur-sm">
        <div className="w-full max-w-2xl bg-white p-6 shadow-xl
animate-fade-in relative max-h-[90vh] overflow-y-auto">
          <button onClick={() => setViewOrder(null)} className="absolute
right-4 top-4 text-gray-400 hover:text-red-500"><FiX size={20} /></button>
          <h3 className="mb-4 font-serif text-xl font-bold
text-primary">Order #{viewOrder.orderId}</h3>

          <div className="grid grid-cols-2 gap-6 mb-6">
            <div>
              <p className="text-xs font-bold uppercase
text-muted">Customer</p>
              <p className="font-medium">{viewOrder.user?.userName}</p>
              <p className="text-sm
text-gray-600">{viewOrder.user?.userEmail}</p>
              <p className="text-sm
text-gray-600">{viewOrder.user?.userPhone}</p>
            </div>
            <div>
              <p className="text-xs font-bold uppercase text-muted">Shipping
Address</p>
              <p className="text-sm text-gray-600
whitespace-pre-wrap">{viewOrder.shippingAddress ||
viewOrder.user?.userAddress || 'N/A'}</p>
            </div>
          </div>

          <div className="mb-6">
            <p className="mb-2 text-xs font-bold uppercase
text-muted">Items</p>
            <div className="border border-gray-100 divide-y divide-gray-100">
              {viewOrder.orderedProducts?.map((p, i) => (
                <div key={i} className="flex items-center gap-4 p-3">
                  <div className="h-12 w-12 bg-gray-100 flex-shrink-0">
```

```
                <img src={getProductImage(p.product?.productId, 48, 48)}
alt={p.name} className="h-full w-full object-cover" />
              </div>
              <div className="flex-1">
                <p className="font-medium text-sm
text-primary">{p.name}</p>
                <p className="text-xs text-muted">Size: {p.size || '-'} · Qty:
{p.quantity}</p>
              </div>
              <p className="font-medium
text-sm">₹{Math.round(p.price)}</p>
            </div>
          ))}
        </div>
      </div>

      <div className="flex justify-between items-center border-t
border-gray-100 pt-4">
        <div>
          <p className="text-xs text-muted">Payment: <span
className="font-semibold
text-primary">{viewOrder.paymentType}</span></p>
          <p className="text-xs text-muted">Date: {new
Date(viewOrder.date).toLocaleString()}</p>
        </div>
        <div className="text-right">
          <p className="text-sm">Subtotal:
₹{Math.round(viewOrder.totalAmount + (viewOrder.discountAmount ||
0))}</p>
          {viewOrder.discountAmount > 0 && <p className="text-sm
text-green-600">Discount:
-₹{Math.round(viewOrder.discountAmount)}</p>}
          <p className="text-lg font-bold text-primary">Total:
₹{Math.round(viewOrder.totalAmount)}</p>
        </div>
      </div>

      <div className="mt-6 flex justify-end">
        <button onClick={() => setViewOrder(null)}
className="bg-primary px-6 py-2 text-xs font-semibold uppercase text-white
hover:bg-accent transition">Close</button>
```

```
          </div>
        </div>
      </div>
    )}
  </div>
 );
};

export default AdminDashboard;
```

PRODUCT DETAILS PAGE:

```
import { useEffect, useState } from 'react';
import { useParams, Link } from 'react-router-dom';
import api from '../api/axios';
import { useCart } from '../context/CartContext';
import { FiShoppingBag, FiHeart, FiChevronLeft } from 'react-icons/fi';

const ProductDetails = () => {
  const { id } = useParams();
  const [product, setProduct] = useState(null);
  const { addToCart } = useCart();
  const [selectedSize, setSelectedSize] = useState('');
  const [selectedColor, setSelectedColor] = useState('');
  const [selectedVariant, setSelectedVariant] = useState(null);

  useEffect(() => {
    const fetchProduct = async () => {
      try {
        const res = await api.get(`/products/${id}`);
        setProduct(res.data);

        // Auto-select first variant
        if (res.data.variants?.length > 0) {
          const first = res.data.variants[0];
          setSelectedSize(first.size);
          setSelectedColor(first.color);
          setSelectedVariant(first);
```

```
      }
    } catch (err) {
      console.error('Error fetching product', err);
    }
  };
  fetchProduct();
}, [id]);

// Resolve variant when size/color change
useEffect(() => {
  if (!product?.variants) return;
  const match = product.variants.find(
    (v) => v.size === selectedSize && v.color === selectedColor
  );
  setSelectedVariant(match || null);
}, [selectedSize, selectedColor, product]);

if (!product) {
  return (
    <div className="flex min-h-[60vh] items-center justify-center">
      <div className="h-8 w-8 animate-spin rounded-full border-2
border-gray-200 border-t-accent" />
    </div>
  );
}

// Curated Unsplash fashion photos (same set as ProductCard)
const FASHION_IMAGES = [
  'photo-1596755094514-f87e34085b2c',
  'photo-1594938298603-c8148c4dae35',
  'photo-1551028719-00167b16eac5',
  'photo-1572804013309-59a88b7e92f1',
  'photo-1509631179647-0177331693ae',
  'photo-1610030469983-98e550d6193c',
  'photo-1583391733956-3750e0ff4e8b',
  'photo-1519238263530-99bdd11df2ea',
  'photo-1614252235316-8c857d38b5f4',
  'photo-1601924582970-9238bcb495d9',
```

```
  ];
  const imgIdx = ((product.productId || 1) - 1) % FASHION_IMAGES.length;
  const imgSrc =
`https://images.unsplash.com/${FASHION_IMAGES[imgIdx]}?w=800&h=1
060&fit=crop&q=80`;

  // Extract unique sizes & colors from variants
  const sizes = [...new Set(product.variants?.map((v) =>
v.size).filter(Boolean))];
  const colors = [...new Set(product.variants?.map((v) =>
v.color).filter(Boolean))];

  // Available colors for currently selected size
  const colorsForSize = [...new Set(
    product.variants?.filter((v) => v.size === selectedSize).map((v) =>
v.color).filter(Boolean)
  )];

  return (
    <div className="animate-fade-in py-10">
      <Link to="/" className="mb-8 inline-flex items-center gap-1 text-xs
font-medium uppercase tracking-widest text-muted transition
hover:text-primary">
        <FiChevronLeft size={14} /> Back to collections
      </Link>

      <div className="grid gap-10 md:grid-cols-2 lg:gap-16">
        {/* Image */}
        <div className="aspect-[3/4] overflow-hidden bg-gray-100">
          <img
            src={imgSrc}
            alt={product.productName}
            className="h-full w-full object-cover"
            onError={(e) => { e.target.src =
'https://images.unsplash.com/photo-1523381210434-271e8be1f52b?w=800&q
=80'; }}
          />
        </div>

        {/* Info */}
```

```
<div className="flex flex-col justify-center">
  {product.brand && (
    <span className="mb-1 text-xs font-semibold uppercase
tracking-widest text-accent">
      {product.brand}
    </span>
  )}
  {product.category && (
    <span className="mb-2 text-xs font-medium uppercase
tracking-widest text-muted">
      {product.category.categoryName}
    </span>
  )}
  <h1 className="font-serif text-3xl font-bold text-primary
sm:text-4xl">
    {product.productName}
  </h1>
  <p className="mt-4 leading-relaxed text-muted">
    {product.productDescription || 'Premium quality garment crafted with
attention to detail and comfort.'}
  </p>

  {/* Pricing */}
  <div className="mt-6 flex items-center gap-4">
    <span className="text-2xl font-bold text-primary">
      ₹{product.productPriceAfterDiscount || product.productPrice}
    </span>
    {product.productDiscount > 0 && (
      <>
        <span className="text-lg text-muted
line-through">₹{product.productPrice}</span>
        <span className="rounded bg-accent/10 px-2.5 py-1 text-xs
font-bold text-accent">
          {product.productDiscount}% OFF
        </span>
      </>
    )}
  </div>
```

```
        {/* Color Selector */}
        {colors.length > 0 && (
          <div className="mt-6">
            <span className="text-xs font-semibold uppercase tracking-widest
text-primary">
              Color — <span className="font-normal
text-muted">{selectedColor}</span>
            </span>
            <div className="mt-2 flex flex-wrap gap-2">
              {colors.map((c) => {
                const isAvailable = !selectedSize || colorsForSize.includes(c);
                return (
                  <button
                    key={c}
                    onClick={() => setSelectedColor(c)}
                    disabled={!isAvailable}
                    className={`min-w-[60px] border px-3 py-2 text-xs
font-medium transition ${
                      selectedColor === c
                        ? 'border-primary bg-primary text-white'
                        : isAvailable
                        ? 'border-gray-200 text-muted hover:border-gray-400'
                        : 'border-gray-100 text-gray-300 cursor-not-allowed'
                    }`}
                  >
                    {c}
                  </button>
                );
              })}
            </div>
          </div>
        )}

        {/* Size Selector */}
        {sizes.length > 0 && (
          <div className="mt-5">
            <span className="text-xs font-semibold uppercase tracking-widest
text-primary">Size</span>
            <div className="mt-2 flex flex-wrap gap-2">
```

```
            {sizes.map((s) => {
              // Check if this size is in stock for any color (or for selected color)
              const variantsForSize = product.variants.filter((v) => v.size === s);
              const hasStock = variantsForSize.some((v) => v.stock > 0);
              return (
                <button
                  key={s}
                  onClick={() => setSelectedSize(s)}
                  disabled={!hasStock}
                  className={`min-w-[44px] border px-3 py-2 text-xs
font-medium transition ${
                    selectedSize === s
                      ? 'border-primary bg-primary text-white'
                      : hasStock
                      ? 'border-gray-200 text-muted hover:border-gray-400'
                      : 'border-gray-100 text-gray-300 line-through
cursor-not-allowed'
                  }`}
                >
                  {s}
                </button>
              );
            })}
          </div>
        </div>
      )}

      {/* Fabric */}
      {product.fabricType && (
        <p className="mt-4 text-xs text-muted">
          <span className="font-semibold uppercase tracking-widest
text-primary">Fabric:</span> {product.fabricType}
        </p>
      )}

      {/* Variant Stock */}
      <p className="mt-3 text-xs text-muted">
        {selectedVariant
          ? selectedVariant.stock > 0
```

```
            ? `${selectedVariant.stock} in stock`
            : 'Out of stock for this variant'
          : sizes.length > 0 ? 'Select size & color' : `${product.totalStock} in
stock`
        }
      </p>

      {/* Actions */}
      <div className="mt-8 flex gap-3">
        <button
          onClick={() => selectedVariant && addToCart(product.productId,
selectedVariant.variantId)}
          disabled={!selectedVariant || selectedVariant.stock <= 0}
          className="flex flex-1 items-center justify-center gap-2 bg-primary
py-4 text-sm font-semibold uppercase tracking-widest text-white transition
hover:bg-accent disabled:bg-gray-300 disabled:cursor-not-allowed"
        >
          <FiShoppingBag size={16} />
          {!selectedVariant ? 'Select Options' : selectedVariant.stock <= 0 ?
'Out of Stock' : 'Add to Bag'}
        </button>
        <button className="flex h-[52px] w-[52px] items-center
justify-center border border-gray-200 text-gray-400 transition
hover:border-accent hover:text-accent">
          <FiHeart size={18} />
        </button>
      </div>

      {/* Details Accordion */}
      <div className="mt-10 space-y-4 border-t border-gray-100 pt-6">
        <details className="group">
          <summary className="cursor-pointer text-xs font-semibold
uppercase tracking-widest text-primary">
            Product Details
          </summary>
          <p className="mt-2 text-sm leading-relaxed text-muted">
            {product.productDescription || 'High-quality fabric. Designed for
comfort and style.'}
          </p>
        </details>
```

```
          <details className="group">
            <summary className="cursor-pointer text-xs font-semibold
uppercase tracking-widest text-primary">
              Care Instructions
            </summary>
            <p className="mt-2 text-sm leading-relaxed text-muted">
              Machine wash cold. Tumble dry low. Do not bleach. Iron on low
heat if needed.
            </p>
          </details>
          <details className="group">
            <summary className="cursor-pointer text-xs font-semibold
uppercase tracking-widest text-primary">
              Shipping & Returns
            </summary>
            <p className="mt-2 text-sm leading-relaxed text-muted">
              Free shipping on orders over ₹999. Easy 15-day returns &
exchanges.
            </p>
          </details>
        </div>
      </div>
    </div>
  </div>
 );
};

export default ProductDetails;
```

PROFILE CODE:

```
import { useEffect, useState } from 'react';
import { useAuth } from '../context/AuthContext';
import api from '../api/axios';
import { toast } from 'react-toastify';
import { FiEdit3, FiSave, FiX, FiPackage } from 'react-icons/fi';

const Profile = () => {
  const { user: authUser } = useAuth();
```

```
const [user, setUser] = useState(null);
const [editing, setEditing] = useState(false);
const [formData, setFormData] = useState({});
const [orders, setOrders] = useState([]);
const [loading, setLoading] = useState(true);

useEffect(() => {
  if (!authUser?.userId) return;
  const fetchData = async () => {
  try {
      const [profileRes, ordersRes] = await Promise.all([
        api.get(`/user/${authUser.userId}`),
        api.get(`/orders/user/${authUser.userId}`),
      ]);
      setUser(profileRes.data);
      setFormData(profileRes.data);
      setOrders(ordersRes.data);
    } catch (err) {
      console.error('Failed to load profile', err);
    } finally {
      setLoading(false);
    }
  };
  fetchData();
}, [authUser]);

const handleSave = async () => {
  try {
    await api.put(`/user/${authUser.userId}`, {
      userName: formData.userName,
      userPhone: formData.userPhone,
      userGender: formData.userGender,
      userAddress: formData.userAddress,
      userCity: formData.userCity,
      userPincode: formData.userPincode,
      userState: formData.userState,
    });
    setUser({ ...user, ...formData });
```

```
      setEditing(false);
      toast.success('Profile updated!');
    } catch (err) {
      toast.error('Failed to update profile');
    }
  };

  if (loading) {
    return (
      <div className="flex min-h-[60vh] items-center justify-center">
        <div className="h-8 w-8 animate-spin rounded-full border-2
border-gray-200 border-t-accent" />
      </div>
    );
  }
  if (!user) return <p className="py-20 text-center text-muted">User not
found</p>;

  const fields = [
    { key: 'userName', label: 'Name', editable: true },
    { key: 'userEmail', label: 'Email', editable: false },
    { key: 'userPhone', label: 'Phone', editable: true },
    { key: 'userGender', label: 'Gender', editable: true, type: 'select', options:
['Male', 'Female', 'Other'] },
    { key: 'userAddress', label: 'Address', editable: true },
    { key: 'userCity', label: 'City', editable: true },
    { key: 'userPincode', label: 'Pincode', editable: true },
    { key: 'userState', label: 'State', editable: true },
  ];

  const statusColor = (s) => {
    if (!s) return 'bg-gray-100 text-gray-600';
    switch (s.toLowerCase()) {
      case 'delivered': return 'bg-green-50 text-green-700';
      case 'shipped': return 'bg-blue-50 text-blue-700';
      case 'cancelled': return 'bg-red-50 text-red-700';
      default: return 'bg-amber-50 text-amber-700';
    }
  };
```

```
  return (
    <div className="animate-fade-in py-10">
      <div className="mx-auto max-w-2xl">
        {/* ── Header ── */}
        <div className="flex items-center justify-between">
          <div>
            <h1 className="font-serif text-3xl font-bold text-primary">My
Profile</h1>
            <p className="mt-1 text-sm text-muted">Manage your account
details</p>
          </div>
          {!editing ? (
            <button onClick={() => setEditing(true)} className="flex
items-center gap-2 border border-gray-200 px-4 py-2 text-xs font-semibold
uppercase tracking-widest text-muted transition hover:border-primary
hover:text-primary">
              <FiEdit3 size={14} /> Edit
            </button>
          ) : (
            <div className="flex gap-2">
              <button onClick={handleSave} className="flex items-center gap-2
bg-primary px-4 py-2 text-xs font-semibold uppercase tracking-widest
text-white transition hover:bg-accent">
                <FiSave size={14} /> Save
              </button>
              <button onClick={() => { setEditing(false); setFormData(user); }}
className="flex items-center gap-2 border border-gray-200 px-4 py-2 text-xs
font-semibold uppercase tracking-widest text-muted transition
hover:border-red-400 hover:text-red-500">
                <FiX size={14} /> Cancel
              </button>
            </div>
          )}
        </div>

        {/* ── Profile Details ── */}
        <div className="mt-8 space-y-4 border border-gray-100 p-6">
          {fields.map((f) => (
            <div key={f.key} className="flex items-center justify-between
```

```
border-b border-gray-50 pb-3 last:border-0 last:pb-0">
            <span className="text-xs font-semibold uppercase tracking-widest
text-muted">{f.label}</span>
            {editing && f.editable ? (
              f.type === 'select' ? (
                <select name={f.key} value={formData[f.key] || ''} onChange={(e)
=> setFormData({ ...formData, [f.key]: e.target.value })}
                  className="w-1/2 border border-gray-200 px-3 py-2 text-sm
bg-white outline-none focus:border-accent">
                  <option value="">Select</option>
                  {f.options.map(o => <option key={o} value={o}>{o}</option>)}
                </select>
              ) : (
                <input type="text" value={formData[f.key] || ''} onChange={(e) =>
setFormData({ ...formData, [f.key]: e.target.value })}
                  className="w-1/2 border border-gray-200 px-3 py-2 text-sm
outline-none focus:border-accent" />
              )
            ) : (
              <span className="text-sm text-primary">{user[f.key] ||
'—'}</span>
            )}
          </div>
        ))}
        <div className="flex items-center justify-between pt-2">
          <span className="text-xs font-semibold uppercase tracking-widest
text-muted">Member Since</span>
          <span className="text-sm text-primary">{user.dateTime ? new
Date(user.dateTime).toLocaleDateString('en-IN') : '—'}</span>
        </div>
      </div>


      {/* ── Order History ── */}
      <div className="mt-10">
        <h2 className="flex items-center gap-2 font-serif text-xl font-bold
text-primary">
          <FiPackage size={18} /> Order History
        </h2>
        {orders.length === 0 ? (
          <p className="mt-4 text-sm text-muted">No orders yet. Start
```

```
shopping!</p>
        ) : (
          <div className="mt-4 space-y-3">
            {orders.map((order) => (
              <div key={order.id} className="flex items-center justify-between
border border-gray-100 p-4 transition hover:border-gray-200">
                <div>
                  <p className="text-sm font-medium
text-primary">{order.orderId}</p>
                  <p className="text-xs text-muted">{new
Date(order.date).toLocaleDateString('en-IN')} · {order.paymentType}</p>
                  {order.orderedProducts?.length > 0 && (
                    <p className="mt-1 text-xs
text-muted">{order.orderedProducts.map(p => p.name).join(', ')}</p>
                  )}
                </div>
                <div className="text-right">
                  <span className={`inline-block rounded-full px-3 py-1
text-[10px] font-bold uppercase tracking-wide ${statusColor(order.status)}`}>
                    {order.status}
                  </span>
                  {order.totalAmount > 0 && (
                    <p className="mt-1 text-sm font-semibold
text-primary">₹{Math.round(order.totalAmount)}</p>
                  )}
                </div>
              </div>
            ))}
          </div>
        )}
      </div>
    </div>
  </div>
);
};

export default Profile;
```

APPENDIX 2

SNAPSHOTS

FIGURE A2.0 HOME PAGE

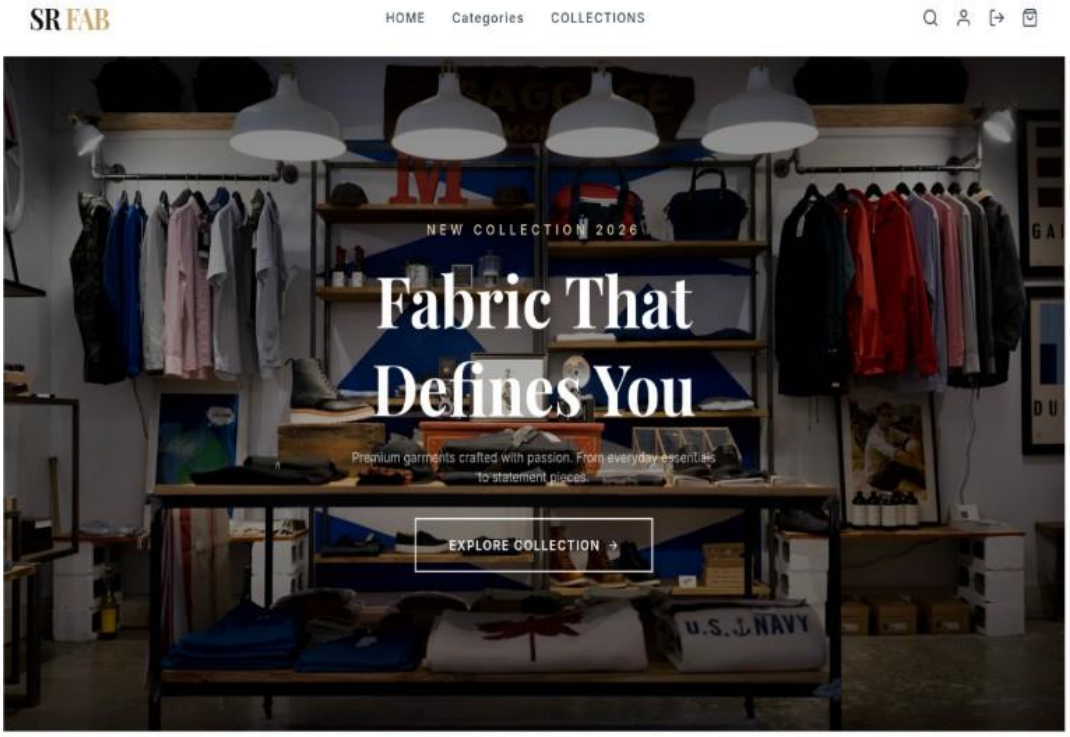


FIGURE A2.1 ADMIN DASHBOARD

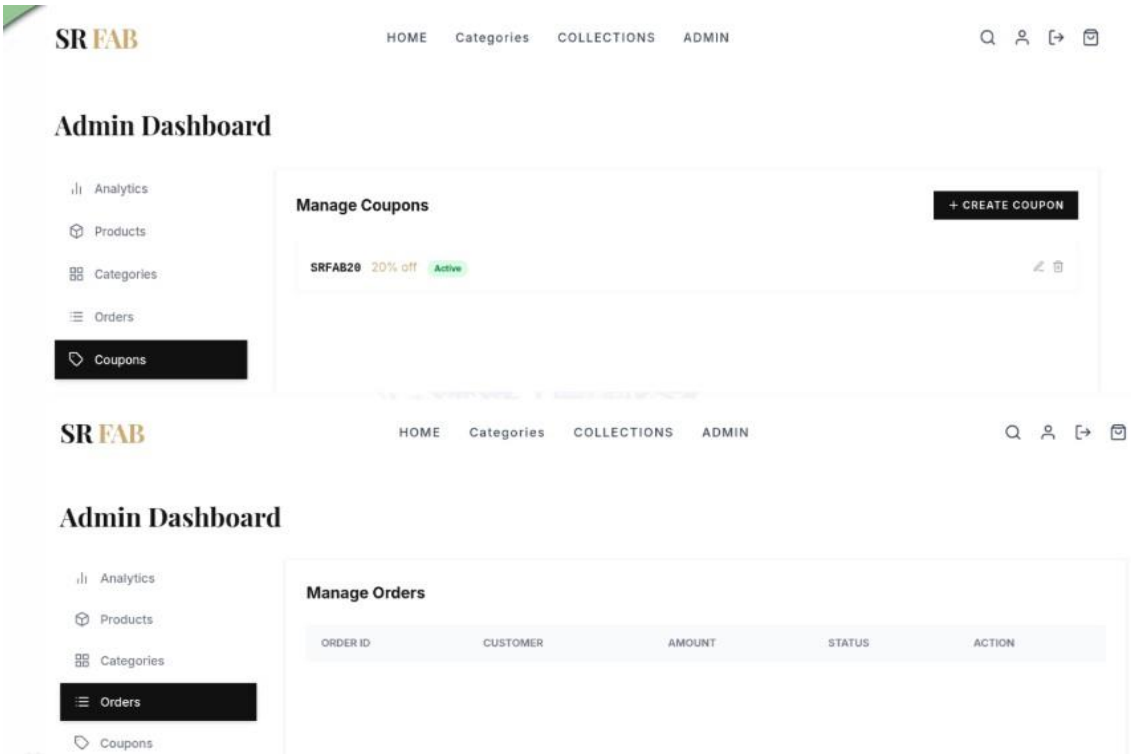


FIGURE A2.2 ADMIN DASHBOARD

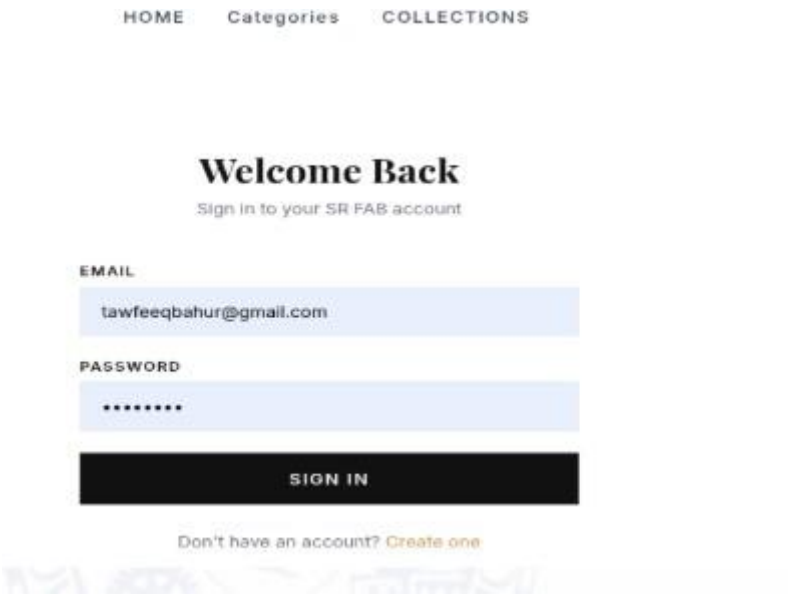
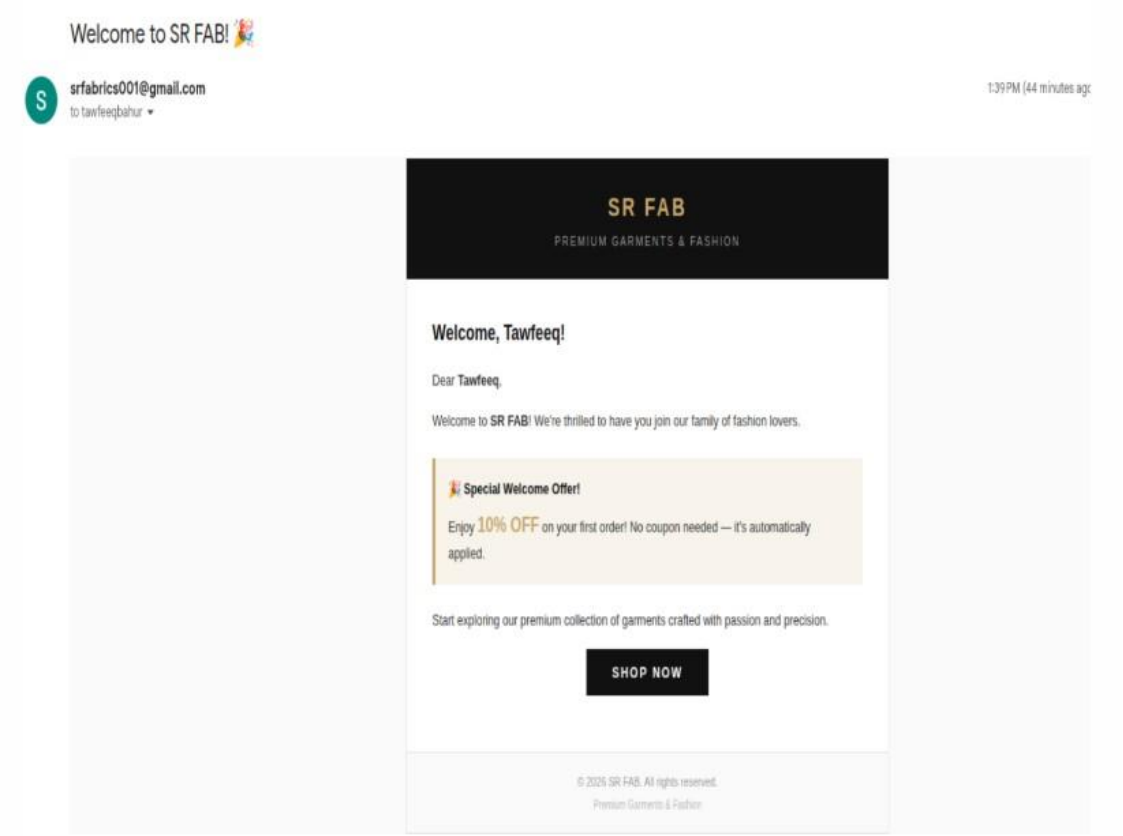


FIGURE A2.3 WELCOME PAGE



REFERENCES

1. Firebase Firestore Documentation. Available at: https://firebase.google.com/docs/firestore
2. Node.js Official Documentation. Available at: https://nodejs.org
3. React Official Documentation. Available at: https://react.dev
4. Tailwind CSS Official Documentation. Available at: https://tailwindcss.com/docs
5. Visual Studio Code Documentation. Available at: https://code.visualstudio.com/docs